CN 2026-03-26

Interpolazione mediante funzioni spline.

Dato $\Delta = \{ a = x_0 < x_1 < \dots < x_n = b \}$ ($n+1$)

diremo che $s_m(x)$ è una spline di grado m su Δ, se:

1) $s_m|_{[x_{i-1}, x_i]}(x) \in \Pi_m$, $\forall i = 1, \dots, n$.

2) $s_m(x) \in C^{(m-1)}[a,b]$.

Ricordiamo che per individuare univocamente una spline di grado m su Δ, occorrono $m+n$ condizioni (indipendenti).

$s_m(x)$ interpola $f(x)$ su Δ se, inoltre:

3) $s_m(x_i) = f(x_i) \equiv f_i$, $i = 0, \dots, n$. ($n+1$)

Queste condizioni permettono di individuare univocamente la spline lineare interpolante.

(lineare $\iff m = 1$).

Al fine di ottenere spline interpolanti che si raccordino "smooth" nei punti di

interpolazione, occorre utilizzare spline di grado più elevato.

Tra queste, le spline più utilizzate nelle applicazioni sono le **spline cubiche** ($m=3$).

Pertanto, in questo caso, per individuare univocamente una spline cubica interpolante una data funzione su Δ, **occorrono $n+3$ condizioni**. Di queste condizioni, **$n+1$** sono le condizioni di interpolazione:

(1) $$s_3(x_i) = f_i, \quad i = 0, \dots, n.$$

Rimangono quindi da imporre **2** ulteriori condizioni: ciascuna scelta di queste 2 condizioni dà origine ad una spline cubica interpolante **diversa**.

Vediamo le scelte più comuni.

Spline cubica naturale:

In questo caso, le due ulteriori condizioni, imposte oltre alle (1), sono:

(2) $s_3''(a) = 0$, $s_3''(b) = 0$ $\quad (a = x_0,\ b = x_n)$.

Spline cubica completa.

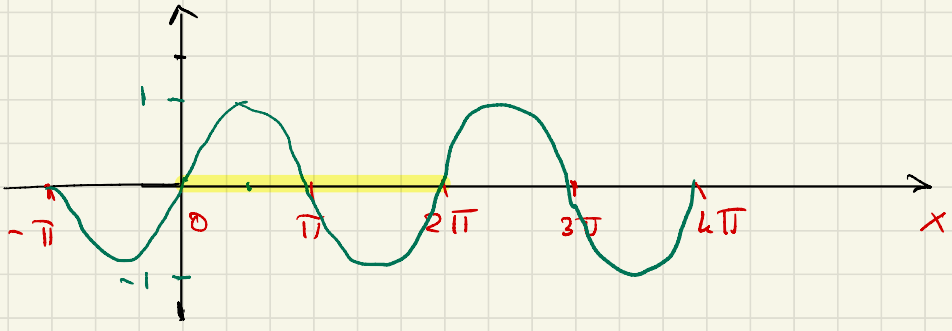
In questo caso, se sono note $f'(a)$ e $f'(b)$, le condizioni aggiuntive, oltre alle condizioni di interpolazione (1), sono:

(3) $s_3'(a) = f'(a)$, $s_3'(b) = f'(b)$.

Spline cubica periodica.

Questa particolare funzione spline ha senso nel momento in cui $f(x)$ è una funzione periodica in $[a, b]$.

Ad esempio, $\sin(x)$, su $[0, 2\pi]$,



In questo caso, abbiamo che

$$\left.\frac{d^j}{dx^j}\sin(x)\right|_{x=0} = \left.\frac{d^j}{dx^j}\sin(x)\right|_{x=2\pi}, \quad \forall j=0,1,\dots$$

Se questo è vero per una generica funzione $f(x)$, periodica in $[a,b]$, avremo, in particolare, assumendo $C^{(2)}[a,b]$, che

(4) $$f^{(j)}(a) = f^{(j)}(b), \quad j=0,1,2.$$

Pertanto, la condizione

$$S_3(a) = S_3(b) \quad [\equiv f(a)=f(b)]$$

deriva già dalle condizioni di interpolazione (1). Le 2 condizioni aggiuntive sono pertanto derivate dalla (4) come:

(5) $$S_3'(a) = S_3'(b), \qquad S_3''(a) = S_3''(b).$$

oss Quindi una spline cubica periodica vuole essere una approssimazione qualitativamente simile alla funzione approssimata.

Spline not-a-knot.

Questa spline, implementata nella function spline di Matlab, è costruita in modo tale che le condizioni di interpolazione (1) siano sufficienti ad individuarla. Le due condizioni aggiuntive si ottengono, implicitamente, imponendo che lo stesso polinomio di grado 3 rappresenti la restrizione della spline in

(6) $[x_0, x_1]$ e $[x_1, x_2]$

e, simmetricamente, lo stesso polinomio di grado 3 rappresenti la restrizione della spline in

(7) $[x_{n-2}, x_{n-1}]$ e $[x_{n-1}, x_n]$.

Ragioniamo per la (6), in quanto la (7) si ottiene per simmetria.

Osserviamo che, per ogni spline cubica, si ottiene che:

(8) $$S_3^{(j)}\Big|_{[x_0,x_1]}(x_1) = S_3^{(j)}\Big|_{[x_1,x_2]}(x_1), \quad j=0,1,2$$

Affinché i due polinomi siano tra loro identici, si dovrà imporre che, oltre alle (8), valga anche:

(9) $$S_3^{(3)}\Big|_{[x_0,x_1]}(x_1) = S_3^{(3)}\Big|_{[x_1,x_2]}(x_1).$$

Essendo la derivata terza di un polinomio di grado 3 una costante, otteniamo che la (9) può essere riscritta, equivalentemente, come:

(10) $$\frac{S_3''(x_1) - S_3''(x_0)}{x_1 - x_0} = \frac{S_3''(x_2) - S_3''(x_1)}{x_2 - x_1}.$$

Simmetricamente, la (7) si declina come:

(11) $$\frac{S_3''(x_{n-1}) - S_3''(x_{n-2})}{x_{n-1} - x_{n-2}} = \frac{S_3''(x_n) - S_3''(x_{n-1})}{x_n - x_{n-1}}.$$

Al fine di ottenere un algoritmo efficiente per il calcolo di una spline cubica interpolante, dobbiamo esaminare un modo efficiente per risolvere un sistema lineare tridiagonale. Si tratta di risolvere il sistema lineare:

nella prossima lezione: vettore x sono le derivate seconde

qui sta spiegando semplicemente il modo di risolvere sistema lineare tridiagonale

(12) $$A\underline{x} = \underline{z},$$

con $\underline{x} = \begin{bmatrix} x_1 \\ | \\ x_n \end{bmatrix}$, $\underline{z} = \begin{bmatrix} z_1 \\ | \\ z_n \end{bmatrix}$, rispettivamente il vettore delle incognite e quello dei termini noti, mentre la matrice dei coefficienti è tridiagonale:

(13) $$A = \begin{bmatrix} a_1 & c_1 & & & \\ b_2 & a_2 & c_2 & & \\ & b_3 & \ddots & \ddots & \\ & & & & c_{n-1} \\ & & & b_n & a_n \end{bmatrix} \in \mathbb{R}^{n \times n},$$

in cui:

b_i, a_i, c_i sono gli elementi della sottodiagonale, diagonale principale, e sopradiagonale sulla riga i-esima ($b_1 = c_n = 0$).

Per memorizzare A, di fatto necessitiamo solo di 3 vettori, che contengono gli elementi delle 3 diagonali.

OSS A è un esempio di matrice sparsa, ovvero una matrice in cui il numero di elementi diversi da 0 è proporzionale a n (= dimensione di A), invece che essere una frazione di n^2 (= numero di elementi di una generica matrice $n \times n$).

[=> help spy di Matlab].

Nel seguito, supporremo che la

matrice A in (12) - (13) sia fattorizzabile LU (ad es. diagonale dominante).

In questo caso, verifichiamo che

$A = LU$ con:

$$L = \begin{bmatrix} 1 & & & \\ \ell_2 & 1 & & \\ & \ell_3 & \ddots & \\ & & \ell_n & 1 \end{bmatrix}, \quad U = \begin{bmatrix} d_1 & c_1 & & \\ & d_2 & c_2 & \\ & & \ddots & c_{n-1} \\ & & & d_n \end{bmatrix}$$

Si tratta, dunque, di derivare la espressione degli ℓ_i e dei d_i.

Esaminiamo, per semplicità, il caso

$n = 3$:

$$\underbrace{\begin{bmatrix} 1 & & \\ \ell_2 & 1 & \\ & \ell_3 & 1 \end{bmatrix}}_{L} \cdot \underbrace{\begin{bmatrix} d_1 & c_1 & \\ & d_2 & c_2 \\ & & d_3 \end{bmatrix}}_{U} = \begin{bmatrix} d_1 & c_1 & \\ \ell_2 d_1 & d_2 + \ell_2 c_1 & c_2 \\ & \ell_3 d_2 & d_3 + \ell_3 c_2 \end{bmatrix}$$

$$\equiv \underbrace{\begin{bmatrix} a_1 & c_1 & \\ b_2 & a_2 & c_2 \\ & b_3 & a_3 \end{bmatrix}}_{= A} .$$

Uguagliando i termini omologhi, otteniamo:

$$d_1 = a_1$$

$$\begin{cases} \ell_i = b_i / d_{i-1} \\ d_i = a_i - \ell_i c_{i-1} \end{cases}, \quad i = 2, 3, \dots, n$$

per un generico $n \geq 3$.

oss

1) possiamo sovrascrivere $\underline{b}$ e $\underline{a}$ con $\underline{\ell}$ e $\underline{d}$ (c rimane invariato).

2) costo della fattorizzazione $\approx 3n$ flops.

Risoluzione per i fattori L ed U:

$$L\underline{y} = \underline{z} \quad \wedge \quad U\underline{x} = \underline{y} \qquad (*)$$

$$\Downarrow$$

$$\begin{cases} y_1 = z_1 \\ y_i = z_i - \ell_i z_{i-1} \end{cases}, \quad i = 2, \dots, n$$

$2n$ flops

inoltre, possiamo sovrascrivere $\underline{z}$ con $\underline{y}$

$$(*) \quad \begin{cases} x_n = y_n / d_n \\ x_i = (y_i - c_i x_{i+1}) / d_i, \quad i = n-1, n-2, \dots, 1 \end{cases}$$

$\approx 3n$ flops

e, al solito, possiamo sovrascrivere $\underline{y}$ con $\underline{x}$.

In conclusione: per risolvere il sistema tridiagonale (12)–(13) occorrono:

a) 4 vettori di lunghezza n;

b) $8n$ flops.

Pertanto, la complessità è lineare.